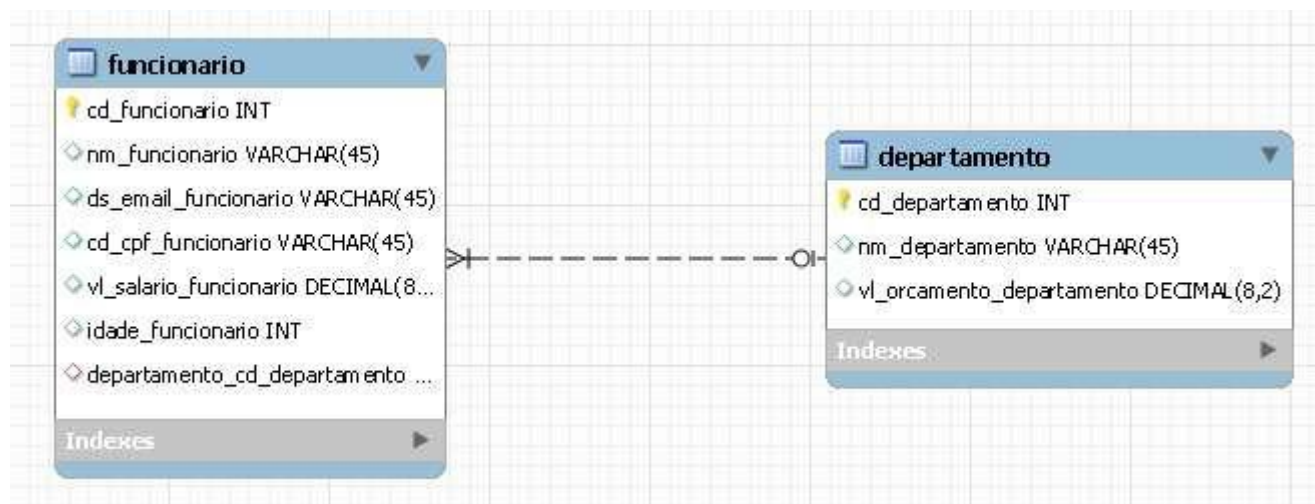


Triggers no SQL

Fonte: <https://www.devmedia.com.br/triggers-no-sql-server-teoria-e-pratica-aplicada-em-uma-situacao-real/28194>

Para este laboratório utilizaremos a proposta de modelo abaixo:



Usando Triggers no SQL

Conforme teoria apresentada em aula, neste tutorial vamos verificar o funcionamento das triggers no SQL; iniciaremos criando nosso modelo de dados inserindo alguns registros no mesmo:

1. CRIAÇÃO DAS TABELAS

Create database rhcc

```
IF OBJECT_ID('auditoria', 'U') IS NOT NULL DROP TABLE auditoria;
IF OBJECT_ID('funcionario', 'U') IS NOT NULL DROP TABLE funcionario;
IF OBJECT_ID('departamento', 'U') IS NOT NULL DROP TABLE departamento;
GO
```

```
CREATE TABLE departamento (
    cd_departamento INT NOT NULL IDENTITY(1,1),
    nm_departamento VARCHAR(45) NULL,
    vl_orcamento_departamento DECIMAL(10,2) NULL,
    PRIMARY KEY (cd_departamento)
);
```

```
CREATE TABLE funcionario (
    cd_funcionario INT NOT NULL IDENTITY(1,1),
    nm_funcionario VARCHAR(45) NULL,
    ds_email_funcionario VARCHAR(45) NULL,
    cd_cpf_funcionario VARCHAR(45) NULL,
    vl_salario_funcionario DECIMAL(10,2),
    idade_funcionario INT NULL,
    cd_departamento INT NULL,
    PRIMARY KEY (cd_funcionario),
    CONSTRAINT fk_funcionario_departamento FOREIGN KEY (cd_departamento)
```

```
REFERENCES departamento (cd_departamento)
);
```

```
CREATE TABLE auditoria (
    cd_auditoria INT NOT NULL IDENTITY(1,1),
    dt_auditoria DATETIME,
    usuario VARCHAR(40) NOT NULL,
    mensagem VARCHAR(100) NULL,
    PRIMARY KEY (cd_auditoria)
);
GO
```

2. POPULANDO DEPARTAMENTOS E FUNCIONÁRIOS

```
INSERT INTO departamento (nm_departamento, vl_orcamento_departamento)
VALUES
('Desenvolvimento', 100000),
('Produto', 200000),
('Serviços', 100000),
('Diretoria', 500000);
```

```
INSERT INTO funcionario (nm_funcionario, ds_email_funcionario, cd_cpf_funcionario,
vl_salario_funcionario,
idade_funcionario, cd_departamento)
VALUES
('Mario', 'mario.quinello@docente.unip.br', '11111111111', 20000, 44, 1),
('Fernando', 'fernando@fernando.com.br', '22222222222', 10000, 24, 3),
('Otavio', 'otavio@otavio.com.br', '33333333333', 30000, 35, 4),
('Lauro', 'lauro@lauro.com.br', '44444444444', 30000, 30, 4),
('Gumercindo', 'gumercindo@gumercindo.com.br', '55555555555', 20000, 50, 1);
GO
```

Verifique se os registros foram inseridos com sucesso:

```
SELECT * FROM funcionario;
SELECT * FROM departamento;
```

Nosso primeiro gatilho será utilizado para monitorar a inserção na tabela funcionário; utilize o script abaixo para criar esta trigger:

```
-- Auditando registros na tabela funcionario - INSERT
```

```
CREATE TRIGGER auditoria_func_ins
ON funcionario
AFTER INSERT
AS
BEGIN
    INSERT INTO auditoria (dt_auditoria, usuario, mensagem)
    SELECT GETDATE(), SYSTEM_USER, 'Novo registro inserido na tabela funcionario!'
    FROM inserted;
END;
GO
```

Para testar a implementação, insira um novo registro na tabela funcionário (deve gerar auditoria):

COMPORTAMENTO ESPERADO: ao adicionar "Humberto", uma linha será registrada na tabela auditoria.

```
INSERT INTO funcionario
(nm_funcionario, ds_email_funcionario, cd_cpf_funcionario, vl_salario_funcionario, idade_funcionario, cd_departamento)
VALUES ('Humberto', 'humberto@humberto.unip.br', '66666666666', 20000, 42, 1);
GO
```

Verifique as tabelas funcionário e auditoria; veja que o registro 1 da tabela auditoria foi disparado pela nossa trigger que armazenou um registro (1) auditando o evento:

```
SELECT * FROM funcionario;

SELECT * FROM auditoria;
```

Agora vamos criar uma trigger para monitorar os registros removidos da tabela funcionarios; neste caso vamos utilizar o conceito de OLD para armazenar a informação sobre qual registro foi removido:

COMPORTAMENTO ESPERADO: ao excluir "Fernando", será registrada mensagem informando qual ID foi removido.

```
-- Auditando registros na tabela funcionario - DELETE
CREATE TRIGGER auditoria_func_del
ON funcionario
AFTER DELETE
AS
BEGIN
    INSERT INTO auditoria (dt_auditoria, usuario, mensagem)
    SELECT GETDATE(), SYSTEM_USER,
           'O registro ' + CAST(d.cd_funcionario AS VARCHAR(10)) + ' foi removido!'
    FROM deleted d;
END;
GO
```

Para testar a trigger, remova o **registro 6** da tabela funcionarios:

```
-- Teste de DELETE (deve gerar auditoria)
DELETE FROM funcionario WHERE nm_funcionario = 'Fernando';
GO
```

Verifique as tabelas funcionário e auditoria:

```
SELECT * FROM funcionario;

SELECT * FROM auditoria;
```

Agora vamos impedir que uma determinada informação seja alterada baseado em uma **regra de negócio implementada via gatilho (Restrição Semântica)**; veja que neste caso a verificação ocorre **antes** que o UPDATE seja efetivado (BEFORE):

“O valor do salário do funcionário não pode ser maior que 5% do orçamento do setor onde atua.”

```
CREATE TRIGGER auditoria_sal_func
ON funcionario
INSTEAD OF UPDATE
AS
BEGIN
    DECLARE @cd_departamento INT, @novo_salario DECIMAL(10,2), @orcamento DECIMAL(10,2);
```

```
SELECT TOP 1
    @cd_departamento = d.cd_departamento,
    @novo_salario = i.vl_salario_funcionario
FROM inserted i
INNER JOIN deleted d ON i.cd_funcionario = d.cd_funcionario;

SELECT @orcamento = vl_orcamento_departamento
FROM departamento
WHERE cd_departamento = @cd_departamento;

IF @novo_salario > (0.05 * @orcamento)
BEGIN
    THROW 50001, 'O registro não foi gravado pois ultrapassa o orçamento previsto!', 1;
END
ELSE
BEGIN
    UPDATE f
    SET nm_funcionario = i.nm_funcionario,
        ds_email_funcionario = i.ds_email_funcionario,
        cd_cpf_funcionario = i.cd_cpf_funcionario,
        vl_salario_funcionario = i.vl_salario_funcionario,
        idade_funcionario = i.idade_funcionario,
        cd_departamento = i.cd_departamento
    FROM funcionario f
    INNER JOIN inserted i ON f.cd_funcionario = i.cd_funcionario;
END
END;
GO
```

Para validar esse gatilho, efetue a atualização abaixo:
-- Teste de UPDATE válido (salário dentro do orçamento)

COMPORTAMENTO ESPERADO: update válido: se o salário for menor que 5% do orçamento, o update roda normal.

```
UPDATE funcionario
SET vl_salario_funcionario = 3000
WHERE nm_funcionario = 'Mario';
GO
```

COMPORTAMENTO ESPERADO: update inválido: vai disparar erro THROW e não atualizar nada.

```
-- Teste de UPDATE inválido (salário ultrapassa orçamento, deve dar erro)
UPDATE funcionario
SET vl_salario_funcionario = 999999
WHERE nm_funcionario = 'Mario';
GO
```

O SGBD não permite a alteração e retorna a informação fornecida na trigger.

Veja que não houve alteração para o funcionário com código 1:

você pode rodar `SELECT * FROM auditoria;` para ver o log de auditoria.

Exercícios:

Com base no modelo utilizado neste tutorial, apresente o script que implementa as triggers abaixo:

- 1- “Verificar no ato de cadastro do funcionário se o mesmo tem idade entre 16 e 70 anos; caso esteja fora dos limites, apontar na tabela auditoria com o respectivo código e retornar a informação se a idade é SUPERIOR ou INFERIOR.”
- 2- “Verificar no ato de cadastro do departamento se o mesmo tem orçamento previsto; caso não tenha, impedir o INSERT e informar o usuário (utilize a função SIGNAL SQLState).”
- 3- “Verificar no ato de cadastro do funcionário se o e-mail do mesmo foi informado; caso não, gravar registro na tabela auditoria informando o código do funcionário que não possui e-mail.”